HTTP healthcheck

Wget и границы проверки

Занятие 16.10.2026. Пишу от своего лица; практика выполнена с подсказками, оформление - с помощью ИИ. Ручной запрос показан на скриншоте; результат автоматического healthcheck подтверждён моим сообщением.

1. Что я изучал

Я продолжил существующий проект lab-compose-pair в ~/compose-pair. В нём web использует training-site:v2, proxy - nginx:1.30.5-alpine. Цель занятия - добавить проверку HTTP внутри web и отличать работающий процесс от успешной проверки приложения.

Wget загружает ресурс по HTTP. Для healthcheck страницу не обязательно сохранять: важен результат выполнения программы. Разобранная команда внутри web:

```
wget -q -T 2 -O /dev/null http://127.0.0.1/
```

- -q включает тихий режим. Отсутствие текста само по себе не доказывает успех.
- -T 2 задаёт сетевой таймаут две секунды, а не гарантированный предел времени всей команды.
- -O /dev/null направляет тело ответа в /dev/null, где данные отбрасываются. Здесь заглавная латинская O.
- http://127.0.0.1/ обращается к корню сайта на стандартном порту 80 в той сетевой среде, где работает wget.

2. HTTP-код и код завершения

HTTP 200 присылает веб-сервер. Код завершения возвращает программа: 0 означает успешное выполнение, ненулевой код - ошибку. Docker оценивает завершение команды проверки; echo "$?" нужен только при ручном наблюдении из оболочки.

Такой wget проверяет успешность загрузки по своим правилам, а не строго единственный HTTP-код 200. Например, он может пройти перенаправление. Он не сравнивает тело ответа с Image version 2.

3. Адрес зависит от места выполнения

```
На VM:          127.0.0.1:8138 -> proxy:80 -> web:80
Внутри web:     127.0.0.1:80   -> сам web
Внутри proxy:   web:80         -> соседний сервис web
```

Loopback 127.0.0.1 внутри web не равен адресу web в сети Compose. Успешный запрос по loopback не доказывает исправность DNS-имени web, связи между контейнерами, настройки proxy или опубликованного порта 8138.

Автоматическая проверка

4. Конфигурация healthcheck

Наставник объяснил и предложил добавить к существующему web:

```
healthcheck:
  test: ["CMD", "wget", "-q", "-T", "2", "-O",
         "/dev/null", "http://127.0.0.1/"]
  interval: 5s
  timeout: 3s
  retries: 3
```

- healthcheck описывает дополнительную проверку здоровья контейнера.
- test задаёт команду. CMD запускает wget напрямую, без промежуточной оболочки; оставшиеся элементы списка - отдельные аргументы. Команды Docker и echo в test не нужны: проверка уже выполняется внутри контейнера.
- interval: 5s - пауза между окончанием проверки и следующей попыткой; длительность самой проверки добавляется к этому времени.
- timeout: 3s - внешний предел Docker для всей одной проверки. Превышение означает неуспех независимо от сетевого таймаута wget.
- retries: 3 - число последовательных неудач для перехода в unhealthy. Успешная проверка сбрасывает счётчик последовательных отказов.

5. Что означают состояния

После запуска статус здоровья - starting. Успешная проверка переводит его в healthy. После трёх последовательных отказов при данном retries он станет unhealthy. Следующий успех вернёт healthy.

Up означает работающий основной процесс контейнера. Healthy означает успешную заданную проверку. Возможен Up (unhealthy). Сам по себе unhealthy не вызывает автоматический перезапуск контейнера обычным Docker.

Переходы starting/unhealthy/healthy были объяснены. Все эти переходы в практике я отдельно не показывал: сообщил только ожидаемый успешный результат. Искусственный отказ в этом занятии не создавался.

6. Новое применение знакомых команд

Я использовал compose exec для запуска проверки внутри работающего web, а config/up/ps - для проверки, применения и наблюдения нового healthcheck. Сохранение YAML само не меняет запущенный контейнер; up может пересоздать изменившийся сервис. Наставник предупредил о сохранении исходного YAML и возможной потере данных только в записываемом слое при пересоздании. Отдельного подтверждения резервной копии YAML я не прислал.

Практика, ошибка и результат

7. Сначала я выполнил запрос на VM

На скриншоте 18:32 виден прямой запуск wget в оболочке linux01 из ~/compose-pair и exit: 4. Каталог проекта не переносит выполнение внутрь контейнера. Адрес назначения был портом 80 VM, а не loopback web.

Для GNU Wget код 4 означает сетевой сбой. Из-за -q подробная ошибка на скриншоте не видна; точную причину сбоя отдельно не установили. Этот результат не доказывает отсутствие интернета и не является HTTP-кодом. Я также отметил, что контейнер нужно сначала запустить. Наставник уточнил второй необходимый шаг - выбрать место выполнения через exec web.

8. Что показано и что сообщено

На скриншоте 18:35 показаны создание сети lab-compose-pair_default, запуск web/proxy и оба контейнера Up. У proxy опубликован 127.0.0.1:8138, proxy.conf подключён только для чтения. Далее выполнено:

```
sudo docker compose exec web wget -q -T 2 -O /dev/null http://127.0.0.1/
echo $?
```

Показан код 0. Это подтверждает ручную успешную HTTP-загрузку внутри web. Точный HTTP-код, содержимое v2 и доступ через proxy этим выводом отдельно не подтверждены.

После инструкции добавить healthcheck и выполнить config/up/ps я ответил: «да, все как ты и сказал». Поэтому web Up (healthy) записан как сообщённый мной результат, а не новый показанный вывод. Итоговый YAML и журнал автоматических проверок не присланы.

9. Самопроверка и помощь

Для таймаута одну секунду я верно назвал замену -T 2 на -T 1. Наставник напомнил, что T - латинская заглавная буква. Это самостоятельная замена знакомого аргумента в рамках блока с опорой, а не независимый диагностический зачёт.

Про границы проверки я сказал, что web подтверждает работу контейнера и его внутреннего IP. Ответ потребовал уточнения: проверен конкретный HTTP-адрес на loopback, а не весь контейнер и не его сетевой адрес в Compose. Новый самостоятельный ответ после уточнения пока не получен.

10. Итог и точка продолжения

Простой healthcheck пройден с помощью по моему сообщению. Ручной wget с кодом 0 показан. Уровни самостоятельности не повышаются; весь модуль Docker не закрыт. Следующее занятие - depends_on: service_started и service_healthy, ожидание готовности при запуске и поведение при последующем отказе. Эта настройка ещё не выполнялась. В этом блоке не подтверждены down и состояние проекта после занятия.

Источники: Docker Compose: healthcheck, Docker: HEALTHCHECK, GNU Wget: коды завершения, BusyBox: wget.